A3: LLM-Guided Grammar-Aware Fuzzing of PostgreSQL

Study guide 1: everything we have built so far, explained (status: 16 October 2026)

Contents

0. How to use this document **3**

1. The project in one page **3**

1.1 Explain the project in five minutes . . . 3

2. Architecture **4**

2.1 Repository map . . . 5

3. The harness (the instrumented server) **5**

4. The oracle: how a bug is recognised **6**

5. The differential plan oracle (finding wrong-result bugs) **7**

6. The grammar language (`.pgg`) - the heart of Stage 2 **7**

6.1 Why a new grammar language? . . . 7

6.2 The language by example . . . 8

6.3 Types . . . 8

6.4 Scopes and the "early" operator . . . 8

6.5 SQL's grouping rules as context flags . . . 9

6.6 Builtins (Python) . . . 9

6.7 Backtracking and termination . . . 9

6.8 Derivation trees, replay and mutation . . . 10

7. The grammar generator (`GrammarGenerator`) **10**

7.1 The random/mutation baseline (Stage 1 generator) . . . 11

8. Schema, configurations and commands **11**

9. Verified results so far (real numbers only) **12**

9.1 What the server accepted (16 October 2026) 13

10. How the project maps to the course **14**

11. Questions the professor may ask (with answers) **14**

12. Glossary **16**

0. How to use this document

This is a learning document, not a submission. It explains **what exists in the repository today, why it was built that way, and how to defend it** in front of the professor. Every number in it was measured on our machine; anything that is not built or not measured yet is marked **(planned)**. Section 11 lists the questions the professor is most likely to ask, with answers.

If you only have 10 minutes: read Sections 1, 2, 9 and 11.

1. The project in one page

Goal. Build a fuzzer for PostgreSQL that generates SQL, runs it on a specially built server, and automatically detects bugs. Use it to

1. **compare three generation strategies scientifically**, under the same time budget, server and schema:

```
Random/Mutation   ->   Grammar-only   ->   Grammar + LLM
(baseline)             (our typed grammar)  (LLM extends the grammar and writes seeds)
```

2. **find at least one real, previously unknown PostgreSQL bug** that the PostgreSQL developers accept (fix commit or confirmation). Full marks for the 50 % implementation component require this.

Grading reminder. 50 % implementation and results, 25 % meetings and final presentation (every member must be able to explain every part), 25 % report in ACM/FSE format. Meeting 2: 26-30 Oct, meeting 3: 2-6 Nov, presentation 12 Nov, paper and repository 13 Nov.

Team. Venu, Bhuvan, and Gowtham. The build was split three ways: the harness, the generators, and the experiment runner. From here, Venu and Bhuvan follow recent PostgreSQL features, Bhuvan also takes each bug candidate, and Gowtham keeps the experiment, the LLM arm, and the paper moving. The professor allows AI assistance; we use it for code, analysis and writing, and we say so.

Why this is hard and interesting. A database accepts only a tiny fraction of random strings, so random fuzzing mostly tests the parser. To reach the planner and executor (where the interesting bugs are), SQL must be *syntactically valid, type-correct and refer to real tables and columns.* And many database bugs do not crash: they silently return the **wrong rows**. So we need both a smart generator and smart **oracles** (bug detectors).

1.1 Explain the project in five minutes

Use this if the professor asks any of us to walk through the work. Study guide 2 is the plan; this is only what already runs.

1. **Problem.** Random SQL almost never reaches the planner. Wrong-result bugs do not crash. We fuzz PostgreSQL `master` with assertions on, because new code has the least testing and an assertion turns silent corruption into a `TRAP:` line.

2. **Method.** Three generators, one executor, one oracle, one time budget. Only the generator changes, so the comparison is fair.
3. **What is built.** A typed grammar (`.pgg`) whose derivation is a typing judgment $\Gamma \vdash e : T$. Each statement keeps its derivation tree. Mutation edits a subtree; replay re-binds names and types, so the mutant stays valid. The oracle catches crashes, assertion failures, internal errors, and wrong results under 17 planner settings.
4. **What is measured.** Selftest 10/10, 65 tests passed, the timing numbers in Section 9. No campaign has run yet, so we do not claim a bug or a coverage number.
5. **What is ours.** One tree is the representation for generation and mutation today, and for metamorphic rewrite and reduction next. The LLM, when we add it, only proposes grammar edits; the server accepts or rejects them.

2. Architecture

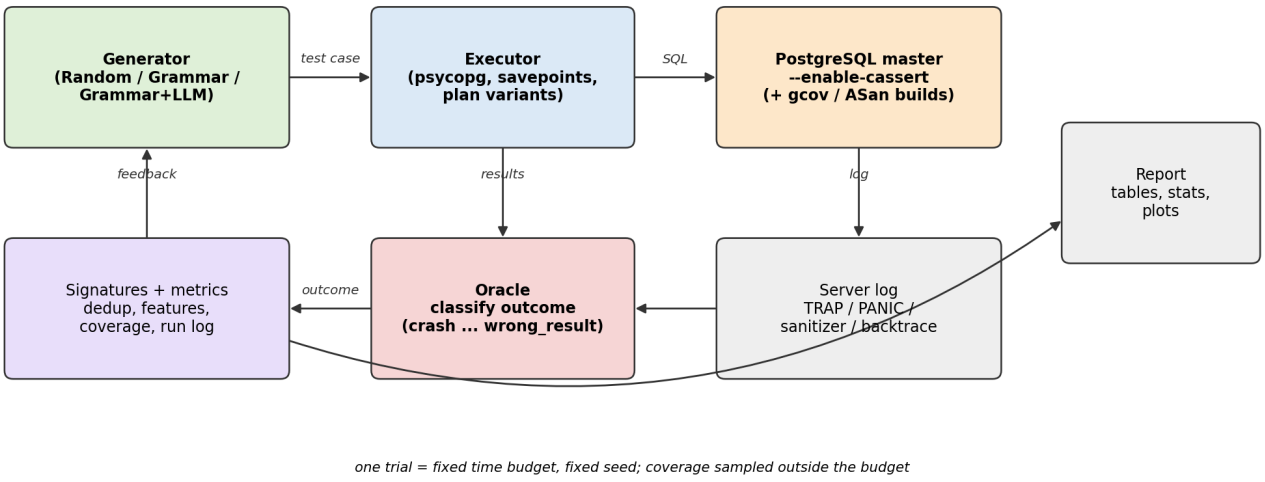


Figure 1: End-to-end architecture. One trial runs one generator against one server for a fixed time budget.

The loop for every test case:

1. The **generator** produces a *test case*: a short list of SQL statements (for example: create an index, then run a query).
2. The **executor** runs them on PostgreSQL inside a savepoint, records each statement's status, SQLSTATE, message and result digest, and re-runs eligible queries under different planner settings (differential oracle). Then it rolls back, so the next test case sees exactly the same database.
3. The **oracle** classifies the outcome from the SQLSTATEs, the connection state and the new lines of the server log (assertion failures, PANICs, sanitizer reports, backtraces).
4. **Signatures** deduplicate bug candidates (two crashes with the same stack are one bug). **Metrics** track valid-statement rate, SQL features covered, coverage samples and unique bugs; everything is written to a run log.

5. **Feedback** goes back to the generator (new features, new signature), which guided generators use to decide what to mutate next.
6. A **report** generator turns many trials into tables, statistics and plots.

2.1 Repository map

```
harness/                          the instrumented server
  docker/Dockerfile               build PostgreSQL from git (assertions, debug, coverage, sanitize
  docker/entrypoint.sh            initdb + server log to a known file
  docker/postgresql.fuzz.conf     fuzzing server settings (fsync off, restart_after_crash on, ...)
  docker/coverage.sh              gcov/lcov coverage as one JSON line, and reset
  docker/gdb_backtrace.sh         crash backtrace: re-run in single-user mode under gdb
  pgctl.py                        manage Docker workers
  local/build_pg.sh               build PostgreSQL without Docker and without root
  local/pg_local.sh               manage N local servers: init/start/stop/coverage/backtrace
src/pgfuzz/                       the fuzzer (Python)
  core/                           data types, plug-in interfaces, SQL splitter
  eval/                           oracle, signatures, features, metrics, differential, statistics
  reference/                      PostgresExecutor, MockExecutor, Random/Corpus generators
  grammar/                        typed grammar language, engine, GrammarGenerator
  runner/                         experiment runner, trials, selftest
  runlog/, report/                run-log format and report generator
src/generator/                    Stage 1 random/mutation baseline (AST seeds, then text mutation)
src/grammar/postgres.pgg          the PostgreSQL grammar
src/corpus/schema.sql, .yaml      the shared database state and its description
src/configs/                      smoke / stage1 / local / hunt experiment configs
src/tests/                        84 unit and end-to-end tests (65 before the baseline was added)
paper/                            the FSE-format paper (LaTeX)
docs/                             these guides
```

3. The harness (the instrumented server)

The harness answers: *on which PostgreSQL do we fuzz, and how do we see what went wrong inside it?*

Decision	Why
Build from **git `master`** (currently `20devel`, commit `d733fca`)	New code has the most undiscovered bugs, and developers fix bugs in unreleased code quickly.
`--enable-cassert` (assertions)	Thousands of internal `Assert()` checks turn silent corruption into a visible `TRAP:` message. Without it, many bugs are invisible.
`--enable-debug`, `-O1` `-fno-omit-frame-pointer`	Readable backtraces with function names for deduplication and for bug reports.
Optional **gcov** build (`COVERAGE=1`)	Line/branch coverage of `src/backend` for the comparison. Counters are reset at the start of every trial and coverage time is excluded from the fuzzing budget.
Optional **ASan/UBSan** build	Memory errors that do not crash immediately.

Decision	Why
Server log to a known file, `log_line_prefix` with pid	The oracle reads only the new log lines after each failed test case.
`restart_after_crash = on`, `fsync = off`	Fast recovery after a crash, fast execution.
`gdb_backtrace.sh`	Re-runs a crashing test case in single-user mode under gdb for a full backtrace.
`build_pg.sh` without root	Our laptop's WSL has no sudo; the script bootstraps m4, bison and flex itself.

Stage 1 as we found it and the gaps we fixed. The original harness built a fixed release (18.6) with coverage only, had hard-coded paths, never reset coverage between trials, could not read the server log, and only detected crashes. We fixed 13 gaps; the most important: assertions on, any git ref, log-based detection, coverage reset, coverage time excluded from the budget, backtrace parsing for stack signatures, sanitizer detection, a new wrong-result oracle, results read as text (so values like `'infinity'::date` are not misreported as errors), a result-list bug after a failed savepoint release, and two Windows performance traps (see Section 9).

4. The oracle: how a bug is recognised

Every statement gets an **outcome**. Ordered from harmless to most severe:

Outcome	Detected by	Bug?
`ok`	success	no
`syntax_error` (42601), `semantic_error` (class 42), `runtime_error` (22, 23, ...)	SQLSTATE class	no
`timeout`, `resource_limit`, `connection_terminated`	statement_timeout, 53xxx, admin shutdown	no
`wrong_result`	differential plan oracle (Section 5)	**yes**
`internal_error`	SQLSTATE `XX000` ("cache lookup failed", "unrecognized node type", ...)	**yes**
`data_corruption`	`XX001` / `XX002`	**yes**
`panic`	`PANIC:` in the log	**yes**
`assertion`	`TRAP: failed Assert(...)` in the log	**yes**
`sanitizer`	AddressSanitizer / UBSan report in the log	**yes**
`crash`	lost connection, "terminated by signal" in the log	**yes**

A test case's outcome is its most severe statement outcome; the **culprit** is that statement. Known-benign internal errors can be filtered by regular expression.

Signatures (deduplication). Many test cases trigger the same bug. A signature is computed with the strongest available evidence:

- `stack`: the top frames of the backtrace, with generic frames (`abort`, `ExceptionalCondition`, `errfinish`, ...) removed;
- `assert`: the assertion text plus file;
- `message`: the normalised error message (numbers, OIDs, names removed);
- `diff`: for wrong results, the plan variant plus the normalised query shape;
- `sanitizer`: sanitizer type plus top frame;
- `weak`: only the statement shape (last resort).

"Unique bugs" in our results are unique signatures. This is a standard, conservative choice: we may under-count (two bugs, same message) or over-count (one bug, two messages), and we say so in the threats to validity.

5. The differential plan oracle (finding wrong-result bugs)

Idea. PostgreSQL's planner settings change *how* a query is executed (join algorithm, scan type, parallelism), never *what* it returns. If the same query returns different rows under two settings, one of the plans is wrong.

Implementation (`src/pgfuzz/eval/differential.py`, executor in `src/pgfuzz/reference/pg_executor.py`):

1. A query is **eligible** only if its result multiset is fully determined by the data: read-only `SELECT/VALUES`, no `LIMIT/OFFSET/FETCH`, no volatile functions (`random()`, `now()`-like clock functions, sequences), no order-dependent window functions (`row_number`, `lag`, ...), no `ROWS` frames, and no order-sensitive aggregate (`string_agg`, `array_agg`, `json_agg`, ...) without its own `ORDER BY`.
2. The executor computes an order-insensitive **digest** of the result (rows as text, sorted). If the query aggregates, float columns are masked, because summation order legitimately changes the last digits.
3. It re-runs the query under up to 2 of **17 plan variants**, chosen deterministically per query: nested loop only, hash join only, merge join only, no hash aggregation, no sort optimisations, index scans only, sequential scans only, no memoize/material, forced parallel query, partition-wise join/aggregate, no partition pruning, GEQO, no join reordering, tiny `work_mem`, no GROUP BY/DISTINCT reordering, no self-join elimination, and JIT. Each variant runs with `set_config(..., true)` inside a savepoint that is rolled back.
4. Different digests give `wrong_result`; an error or crash *only* under a variant is reported with the variant's settings, so it can be reproduced.

Why the grammar cooperates. Our grammar avoids generating non-deterministic constructs where possible; for example `string_agg` and `array_agg` are generated with `ORDER BY` on their own argument (`@sorted_agg`), so their results are plan-independent.

6. The grammar language (`.pgg`) - the heart of Stage 2

6.1 Why a new grammar language?

Plain context-free grammars (like the official `gram.y`) produce syntactically valid SQL that is almost always **semantically invalid**: columns that do not exist, type mismatches, aggregates in `WHERE`, ungrouped columns. PostgreSQL rejects it in the analyser, so the planner and executor are never tested. We need a grammar that knows **types, scopes and SQL's grouping rules**, while staying a readable grammar file that the LLM can edit (Stage 3).

6.2 The language by example

```
%type int    "int"     num ord key mm brin shape w=6     # type, SQL name, classes, weight
%type text   "text"    ord key mm brin spg shape w=4

expr<T>                                  # generic rule with type parameter T
  : <8> @col(T)                          # weight 8: a visible column of type T
  | <4> @lit(T)                          # a literal of type T (edge values included)
  | <1> CASE WHEN expr<bool> THEN expr<T> ELSE expr<T> END
  | <3 ?agg_ok> agg<T>                   # only where aggregates are allowed
  ;
expr<T:num> : <2> "(" expr<T> ("+" | "-" | "*") expr<T> ")" ;   # only numeric types
expr<bool>  : <4> "(" expr<X:key> cmp expr<X> ")" ;             # X: a local type variable
```

Construct	Meaning
`KEYWORD`, `"text"`	emitted literally (punctuation is quoted)
`name<type>`	non-terminal with a type argument
`T`, `int`, `num`, `X:key`, `*`	the rule's parameter, a concrete type, a fresh member of a class, a local type variable bound at first use, any type
`@name(args)`	builtin written in Python (columns, tables, literals, scopes, ...)
`$T`	SQL spelling of a type variable (for casts)
`x?`, `x?0.2`, `x*`, `x+`, `x{1,3/","}`	optional, repetitions, separated lists
`^x`	expand `x` *before* the other elements, but render it in place
`<w ?flag ?!flag>`	alternative weight and required/forbidden context flags
`@scope`, `@scope(isolated)`, `@scope(lateral)`	the rule opens a new query level
`@set(f)`, `@clear(f)`, `@shape`, `@frame`	rule attributes: flags, output shape, join frame

6.3 Types

Thirteen SQL types: `int`, `bigint`, `numeric`, `float8`, `text`, `boolean`, `date`, `timestamptz`, `interval`, `jsonb`, `int4range`, `text[]`, `int[]`. Classes group them: `num` (arithmetic), `ord` (orderable), `key` (equality/hash), `mm` (has `min`/`max`), `arr`, `brin`, `spg`, `shape` (allowed in select lists). A request `expr<int>` collects all variants that match `int`: the generic `expr<T>`, the class variant `expr<T:num>` and the specific `expr<int>`.

Course link (L29-L30, type systems): generation is **type-directed**. Producing `expr<T>` is like constructing a derivation of the typing judgment $\Gamma \vdash e : T$, where Γ (the scope) holds the visible columns and their types. The engine searches for such a derivation with backtracking.

6.4 Scopes and the "early" operator

SQL's text order is not its dependency order: the select list is written before `FROM`, but can only mention tables from `FROM`. The `^` marker solves this:

```
select_core
  : SELECT @items ^from_clause ^where_clause?
  | SELECT @items ^from_clause ^where_clause? GROUP BY ^@group_keys having_clause?
  ;
```

`FROM`, `WHERE` and the grouping keys are expanded first (in that order), then the select list, but the SQL is rendered in normal order. Each `@table` binds a unique alias (`a1`, `a2`, ...) in the current scope; `@col(T)` picks a visible column of type `T` and renders it qualified (`a1.c0`), so names are never ambiguous.

- `@scope` opens a correlated level (subqueries in `WHERE` may use outer columns);
- `@scope(isolated)` is for derived tables, CTE bodies, views (no outer columns);
- `@scope(lateral)` is for `LATERAL` subqueries;
- `@frame` restricts a `JOIN ... ON` condition to the join's own inputs, exactly as PostgreSQL requires.

6.5 SQL's grouping rules as context flags

- After `@group_keys`, the query level is **grouped**: plain column references are restricted to the grouping columns.
- `agg_ok` (aggregates allowed) is set in the select list of a grouped query and in `HAVING`; never in `WHERE`, `ON` or `GROUP BY`.
- Inside an aggregate, `in_agg` lifts the restriction (any column may be aggregated) and `agg_ok`/`win_ok` are cleared (no nested aggregates or windows).
- `win_ok` allows window functions only in the select list.

Grouping variants include `ROLLUP`, `CUBE`, `GROUPING SETS` and global aggregates (no `GROUP BY`).

6.6 Builtins (Python)

`@table`, `@col`, `@lit`, `@items` (typed select list, `AS c0, c1, ...`), `@ordinal` (`ORDER BY 1`), `@group_keys`, `@global_agg`, `@derived_alias`, `@cte_name`, `@define_alias` (for set-returning functions), `@define_rel` (recursive CTE), `@join_eq` (hash/merge-joinable equality between the two join inputs), `@sorted_agg`, `@shape_list`, `@out`, `@target`, `@target_col`, `@insert_body`, `@assign`, `@new_table`, `@new_col`, `@newrel_col`, `@fill` (fills a new table with `generate_series`), `@target_name`, `@fresh_name`.

Shapes. A derived table, CTE, set operation (`UNION` ...), `IN` subquery or `CREATE TABLE AS` first chooses a *shape* (a list of column types); every branch of the query then produces exactly that shape, so `UNION` branches and `IN (SELECT ...)` always match in arity and type.

6.7 Backtracking and termination

If an alternative cannot be completed (no column of the requested type, depth limit), it raises `Fail`; the engine restores the scope (and the name counters) and tries another alternative. Termination: the engine pre-computes the **minimal derivation depth** of every rule/type and alternative, excludes alternatives that cannot finish within the depth limit (default 14), and beyond a soft depth strongly prefers short alternatives.

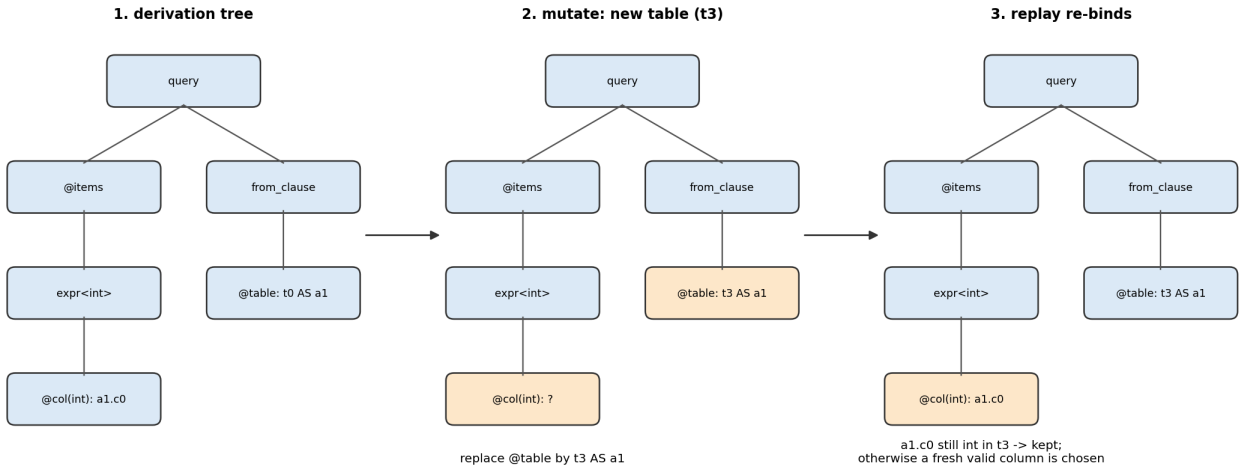


Figure 2: Mutation by replay: a subtree is changed or dropped, and replaying the tree re-binds every name and type around it.

6.8 Derivation trees, replay and mutation

Every generated statement is stored as a **derivation tree** (rule, alternative, children, and every builtin choice such as the chosen column). The tree can be **replayed** as a guide: at each position the recorded choice is reused *if it is still valid in the current context*; otherwise a fresh valid choice is made. This gives mutations that never produce dangling references:

- **regen**: drop one subtree (it is regenerated fresh);
- **splice**: replace a subtree by a subtree with the same rule and type from another interesting test case (Nautilus-style);
- **insert / drop**: add or remove whole statements.

A unit test checks that replaying an unchanged tree reproduces the identical SQL, and that mutated trees replay without errors.

7. The grammar generator (`GrammarGenerator`)

- A **test case** = 0-2 *setup* statements (index, table, view, statistics, `ALTER TABLE`, planner setting, DML) followed by 1-2 statements (mostly queries; also `INSERT`, `UPDATE`, `DELETE`, `MERGE`, `EXPLAIN`). Tables created by setup statements are visible to the later statements.
- **Corpus.** A test case is kept if it produced a new bug signature, a new SQL feature, or (with at least one successful statement) a new *pair* of features. Entries that produce finds are picked more often.
- **Mutation probability** 0.5: half fresh test cases, half mutants.
- **Statistics** for tuning (`stats()`, stored in each trial's summary): outcome counts, valid-statement rate, top 25 normalised error messages, and the 25 grammar alternatives with the lowest success rate (`rule:line`).

What the grammar covers today. Joins of all kinds (including self-joins on keys, `LATERAL`, `USING`), derived tables, scalar/`IN`/`EXISTS`/`ANY`/`ALL` subqueries, CTEs (`MATERIALIZED`, recursive with

SEARCH/CYCLE), UNION/ INTERSECT/EXCEPT, VALUES, GROUP BY/ROLLUP/CUBE/GROUPING SETS, HAVING, window functions with frames, SQL/JSON (JSON_TABLE, JSON_QUERY, JSON_EXISTS, JSON_VALUE, IS JSON), jsonb, ranges, arrays, dates and intervals with edge values (infinity, BC dates, NaN); INSERT ... ON CONFLICT, UPDATE ... FROM, DELETE ... USING, MERGE (including NOT MATCHED BY SOURCE and RETURNING merge_action()), RETURNING old/new; setup DDL: btree/hash/GIN/GiST/BRIN/SP-GiST, partial, expression and INCLUDE indexes, range/hash/list partitioned tables, **virtual and stored generated columns**, identity columns, CREATE TABLE AS, views, materialized views, extended statistics, ALTER TABLE variants and planner settings.

Example of generated SQL (offline generation, seed 3, not yet executed):

```
CREATE TABLE n1 (c0 int, c1 int);
INSERT INTO n1 (c0, c1) SELECT x, CASE WHEN x % 11 = 0 THEN NULL ELSE (x % 17) END
  FROM generate_series(1, 5) AS g(x);
ANALYZE n1;
SELECT (SELECT a1.c3 FROM t3 AS a4 INNER JOIN t4 AS a5 ON ((a4.c0 = a5.c0) AND a4.c1) ...
```

7.1 The random/mutation baseline (Stage 1 generator)

This is the first arm of the comparison. It lives in src/generator/ and is the piece that was easy to leave out of an early copy of the tree. It is wired in now, as generator.random_generator.generator:RandomGenerator in configs/stage1.yaml.

What it does, on purpose:

- Seeds are built with a small SQL AST (generator/common/ast_nodes.py) so the seed itself is valid SQL. The AST covers SELECT, INSERT, UPDATE, DELETE, and CREATE TABLE.
- Those seeds are then mutated as **text** (flip a character, delete a chunk, splice a keyword, swap an operator, drop a clause). The mutators do not know types or scopes. Many mutants are invalid. That invalid rate is the number the grammar arm has to beat.
- When the experiment runs it, randomness comes from the trial seed (ctx.rng), so two trials with the same seed match. feedback is ignored. This arm is unguided. The grammar arm is the one that keeps a corpus.
- Seeds for a real run come from corpus/schema.yaml (tables t0, t1, and the rest). A small demo schema (users, orders, products) remains for the unit tests, which do not need a server.

The stand-in random generator in pgfuzz/reference/generators.py is still there for the old pipeline tests. The experiment does not use it.

8. Schema, configurations and commands

Shared database state (src/corpus/schema.sql), loaded once per connection; every test case runs on top of it and is rolled back:

Table	Purpose
`t0 (int, text, float8)`	NULLs and finite floats (`0.0`, `-1.5`); btree and partial expression index. Earlier samples also seeded `NaN` and `-Infinity`
`t1 (int PK, int FK->t1, jsonb, int4range)`	keys for self-joins; GIN and GiST indexes
`t2 (bigint identity, numeric CHECK, timestamptz, text[])`	identity, BRIN
`t3 (int NOT NULL, bool, date, interval, int[], text COLLATE "C")`	more types, GIN on array
`t4 (int, int VIRTUAL generated, text STORED generated, numeric)`	generated columns (new in PG18)
`tp` range-partitioned, with a DEFAULT partition; `tl` list-partitioned with NULL and DEFAULT partitions	partition pruning and partition-wise plans
`ti` with child `ti_child` (inheritance), view `v0`	inheritance and views

`src/corpus/schema.yaml` describes the same tables for the generator.

Configs. `smoke.yaml` (simulated server, pipeline test only - never report its numbers), `stage1.yaml` (random baseline on a real server), `local.yaml` (development), `hunt.yaml` (4-hour grammar campaigns on `master`, 2 servers).

Commands (`cd src`, then `python -m pgfuzz ...`):

Command	Purpose
`run CONFIG [-j N]`	run an experiment (resumable) and build the report
`report DIR`	rebuild tables, statistics and plots
`selftest CONFIG`	10 acceptance checks of the executor on a real server
`replay CONFIG FILE.sql [--repeat 3]`	re-run a test case and print the oracle verdict
`sample CONFIG -n N [--execute]`	print generated SQL, or run it and print validity statistics
`features FILES [--missing]`	SQL feature coverage of files

9. Verified results so far (real numbers only)

Fact	Value
PostgreSQL built from source	`20devel`, commit `d733fca`, assertions + debug, in WSL Ubuntu 24.04 without root (about 13 minutes on 20 cores)
Executor acceptance test	**10/10** checks passed on that server
Unit and end-to-end tests	**65 passed** when the harness and grammar landed (58 + 7). **84 passed** after the random/mutation baseline and its tests were added

Fact	Value
Executor speed, 20 small test cases	0.15 s without reading the server log, 0.99 s with it - hence the log is now read only after a failure
Connection start from Windows	131 s with `localhost` (IPv6 attempted first) vs 0.2 s with `127.0.0.1`
Grammar size	13 types; about 100 rules; replay reproduces SQL exactly in tests

9.1 What the server accepted (16 October 2026)

Four rounds of `sample configs/local.yaml -n 300 --execute --seed 1` on the assert-enabled `20devel` server. Same seed, same grammar family, after each round of fixes.

Round	Statements	Accepted	Rate	Semantic errors	Time	Bug candidates
1	812	677	83.4%	76	16.7 s	0
2	828	688	83.1%	39	82.9 s	0
3	929	821	88.4%	28	56.1 s	0
4	966	966	**100%**	0	22.1 s	0

Round 1 could not start until we moved index `i6` off `t4.c1`. That column is `VIRTUAL` generated, and this server rejects an index on it. The index is now on `t4.c2`, the stored generated column.

What the later rounds changed, because the error list said so: a grouped query may no longer see columns from an outer query; index `WHERE` clauses are plain comparisons (PostgreSQL requires index predicates to be immutable); `generate_series` no longer uses a table column as its end, which was the source of the timeouts; huge integer literals and multidimensional array literals were removed; `MERGE` emits one `WHEN` clause so a later unconditional `WHEN` is not dead.

Round 4 is the same command after another pass on the grammar, the engine, and the schema. Every statement was accepted. Runtime errors, semantic errors, timeouts, and syntax errors were all zero. The differential oracle checked 230 queries and ran 460 variants, with no mismatch. Corpus 132, generation failures 0, 22.1 seconds.

What that pass changed, in plain words. An aggregate no longer looks up columns in an outer query, so a subquery inside `count(...)` is not blamed for a missing `GROUP BY`. A join equality is taken from the two ends of the join, so `FULL JOIN` has a real key. Text literals are written `::text`, so functions such as `jsonb_build_object` do not see an unknown type. Inserts and updates stay inside NOT NULL, primary keys, the `t2.c1 >= 0` check, foreign keys, and partition keys. `MERGE` reads a one-row source, so it cannot touch the same row twice. JSON functions that need an object are given `jsonb_build_object(...)`. Ranges are built with `LEAST` and `GREATEST`, so the lower bound is not above the upper bound.

Two schema changes are part of the same 100%, and they should be said out loud. `t0` is no longer seeded with `NaN` and infinities, so "cannot subtract infinite dates" cannot come from those seed rows. `tp` has a DEFAULT partition, so an integer key has somewhere to land. The foreign key on `t1.c1` is `ON DELETE SET NULL`. The 100% is this schema, this seed, and these 300 test cases. It is not a claim that every possible SQL statement would be accepted, and it is not a bug.

Not done yet. No multi-hour campaign, so no bug candidate yet. The Docker image has not been built. The LLM arm, the minimizer, and the metamorphic oracles are still the plan in Study guide 2. Six statements in round 3 still hit the 5 second statement timeout.

10. How the project maps to the course

Lecture	Where it appears
L2 static/dynamic/hybrid analysis	Dynamic analysis (fuzzing) with assertion and sanitizer instrumentation; planned hybrid step: static mining of recent commits to direct fuzzing
L3 secure coding	Sanitizer builds; crashes and memory errors are security-relevant
L4 soundness/completeness	The grammar aims to be sound for its fragment (a derivation should be SQL the server accepts) and is incomplete on purpose (not all of SQL). The oracle is sound modulo the determinism rules and incomplete (many bugs cause no mismatch). Both claims are things we measure, not slogans
Hoare logic (check the lecture number on the syllabus)	Planned: each metamorphic relation is an assertion the server must satisfy, namely that the original query and the rewritten query return the same multiset
L5 precision/recall	Oracle precision measured by triage; deduplication over/under-counting discussed
L6 AST/CFG/IR	Derivation trees are our program representation; mutation works on them
L13 coverage and mutation testing	gcov line/branch coverage as a metric; planned bug re-injection benchmark
L14-L16 fuzzing, black/grey-box, grammar-based	The three arms: black-box random, grammar-based, guided by feedback
L29-L30 type systems	Type-directed generation with type classes and local type variables

11. Questions the professor may ask (with answers)

Q1. Why fuzz `master` instead of a release? New code has the most undiscovered bugs, and bugs in unreleased features are fixed quickly and credited ("Reported-by") in the commit. A bug we find in `master` can still be checked against the latest release.

Q2. Why build with assertions? Many bugs corrupt internal state without crashing. Assertions turn them into a `TRAP:` message with file and line, which our oracle reads from the server log.

Q3. How do you avoid false positives in wrong-result detection? Only deterministic queries are compared (no `LIMIT`, volatile functions, order-dependent windows, unordered `string_agg`); results are compared as multisets; float columns are masked in aggregating queries; every candidate is replayed several times before reporting.

Q4. How is this different from SQLsmith and SQLancer? SQLsmith generates random type-aware queries from catalog information and finds mostly crashes; SQLancer focuses on logic-bug oracles (PQS, NoREC, TLP) with hand-written generators. We contribute (a) an editable, typed, scope-aware grammar language with replayable derivation trees, (b) an LLM loop that edits that grammar for new PostgreSQL features (planned), and (c) a controlled three-way comparison. (Verify citations before using them in the paper.)

Q5. Why not use PostgreSQL's own grammar (`gram.y`)? It describes syntax only; random derivations are almost never semantically valid. Our grammar is smaller but typed and scoped, so most statements reach the planner.

Q6. What does the LLM add that a grammar cannot? Knowledge of new features and their corner cases, read from commit messages, documentation and regression tests. It proposes grammar edits and seeds; the grammar then generates thousands of variants cheaply, so the LLM is not on the hot path.

Q7. How do you make the comparison fair? Same server build, schema, executor, oracle and time budget; 10 trials per arm with different seeds; coverage time excluded from the budget; non-parametric statistics (Mann-Whitney U, Vargha-Delaney A12).

Q8. What is a "unique bug"? A unique signature: stack frames, assertion text, normalised message, or plan variant plus query shape. Final counts are confirmed by manual triage.

Q9. How do mutations stay valid? Mutations act on derivation trees; replay re-validates every recorded choice in the new context and replaces invalid ones (Section 6.8).

Q10. What if you do not find a new bug? Two different grades, answered separately.

- The **report** (25 %) and the **meetings** (25 %) do not require a new bug. RQ1 (validity and coverage), RQ3 (time to detect re-injected historical bugs) and RQ4 (what the LLM added, and the token cost) are complete results on their own. If we have no new bug, the paper's motivating example is one of those re-injected bugs, with the measured time for each arm. RQ2 then says zero new bugs, and we report how many candidates we rejected and why. That is a precision result, not a hole.
- The **implementation** component (50 %) reaches its top only with a previously unknown bug the developers accept: a fix, or an acknowledgement on the mailing list. We file anything plausible in week 2, because an acknowledgement before 13 November is the target we can still hit. We do not describe a missing bug as a success on that 50 %.

Q16. What is original, as opposed to SQLsmith plus an LLM? Three claims, two of them already in the code. (1) One typed derivation tree is the representation for generation, mutation, and, next, rewrite and reduction, so a change cannot break types or scopes. (2) We state soundness for the generator and the oracle and we measure both (valid-statement rate, triage precision). (3) The LLM never sits on the hot path and never contributes a number to the paper unless the server runs the proposal and the changed lines gain coverage.

Q11. Why savepoints instead of a fresh database per test case? Speed: rollback to a savepoint takes microseconds; recreating the database takes seconds. The selftest verifies that no state leaks between test cases.

Q12. How do you reproduce a crash? `replay` re-runs the test case; `pg_local.sh backtrace` runs it in single-user mode under gdb; plan-variant bugs are reproduced with the recorded settings.

Q13. Why WSL and not Docker? Our laptop has no Docker; the local scripts do the same without root. The Dockerfile is kept for parallel coverage experiments on other machines.

Q14. Is using AI allowed? Yes; the professor's updated instructions allow it fully. We use it for code, analysis and writing, and disclose it in the paper.

Q15. What is the main risk? Not finding a new bug in time. Mitigations: fuzz new features in `master`, multiple oracles, long campaigns, fast triage. The latest grammar sample accepted every statement, 966 of 966 (Section 9.1), so the next hours are spent hunting, not fighting the parser.

Q17. What did the server actually accept? Three rounds of 300 grammar test cases, seed 1, on the assert-enabled `20devel` server. Accepted statements: 83.4%, then 83.1%, then 88.4% (821 of 929), then **100%** (966 of 966, 22.1 seconds, no errors). Semantic errors fell from 76 to 0. Bug candidates: zero. The differential oracle checked 230 queries on that last sample. Round 1 could not start until index `i6` was moved off a virtual generated column. The random/mutation baseline (Section 7.1) is in the tree and uses the same schema; it has not had its own multi-hour run yet.

12. Glossary

Term	Meaning
Test case	a short list of SQL statements run together
Oracle	the component that decides whether an execution reveals a bug
Differential testing	comparing two executions that must agree
Metamorphic testing	comparing a query with a transformed query whose result relation is known
Signature	a fingerprint used to deduplicate bug candidates
Assertion (cassert)	an internal consistency check compiled into the server
Savepoint	a nested transaction point; rolling back undoes the test case
Derivation tree	the record of how the grammar produced a statement
Replay	regenerating a statement from a (possibly mutated) derivation tree
Shape	the list of column types a (sub)query must produce
Valid-statement rate	fraction of generated statements that execute without error
A12	Vargha-Delaney effect size: probability that one arm beats another